Take-home 2

Submit Assignment

Due Thursday by 11:59pm **Points** 100 **Submitting** a file upload **File Types** py
Available until Oct 21 at 11:59pm

ENGI E1006 - Intro to Computing for Engineers & Applied Scientists

Take-home project 2

Due: Thursday October 17 11:59pm
Total Points: 100

You are welcome to discuss the problems with other students but the code you submit for take-home projects must be your own work. Please review the academic honesty policy for this course (at the end of the **syllabus page**).

Do not use Python modules other than those explicitly mentioned.

Upload the following file to Courseworks:

project2.py

Important: If you re-submit your assignment you need to re-upload **all** files, even if you changed just one of them. Otherwise, any files you uploaded previously will be lost if you submit a second time

A Dialog System for Ordering Pizza

A dialog system (or conversational agent) is a computer program intended to communicate with humans in natural language, following a specific dialog structure to accomplish a specific tasks. Common examples are automatic phone systems for flight reservations or personal assistants such as Siri and Alexa. In this problem we will use functions to write a **very** basic dialog system (with limited vocabulary). The system should allow the user to order from an Italian restaurant by asking specific questions. Here is an example Dialog:

Python prints: "Hello, would you like pizza or salad?"
User types: "pizza"
Python: "Small, medium, or large?"
User: "large"
Python: "Add a topping: pepperoni, mushrooms, spinach, or say 'done'"
User: "mushrooms"
Python: "Add a topping: pepperoni, mushrooms, spinach, or say 'done'"
User: "spinach"

Python: "Add a topping: pepperoni, mushrooms, spinach, or say 'done'"
User: "done"
Python: "You ordered a large pizza with mushrooms and spinach. Place another order or say 'done'.

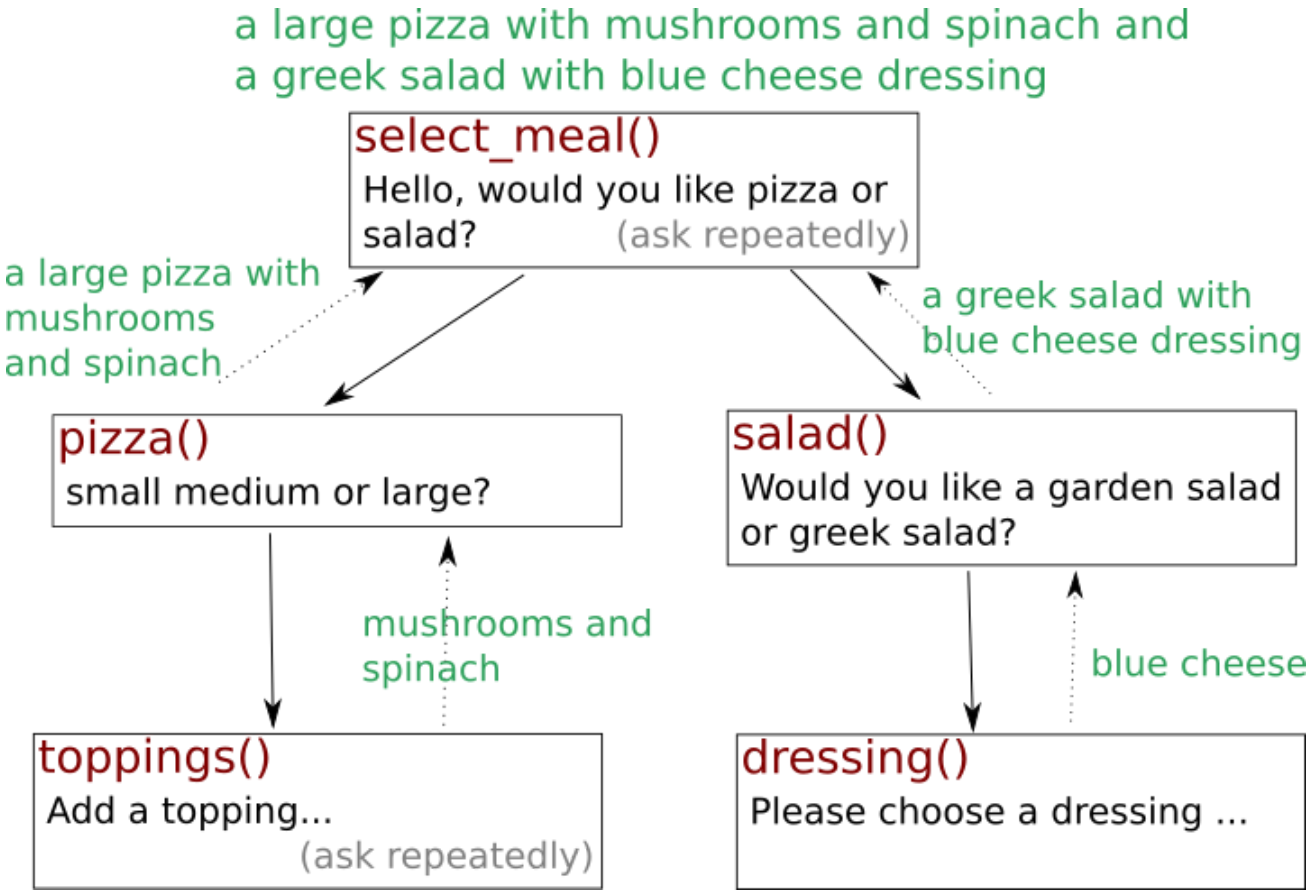
Hello, would you like pizza or salad?"
User: "salad"
Python: "Would you like a garden salad or greek salad"
User: "greek"
Python: "please choose a dressing: vinaigrette, ranch, blue cheese, lemon"
User: "blue cheese"
Python: "You ordered a large pizza with mushrooms and spinach and a greek salad with blue cheese dressing. Place another order or say done'.
Hello, would you like pizza or salad?"
User: "done"
Python: "Your order has been placed. Goodbye."

Implement functions to prompt the user, as illustrated in the diagram below. For example, the salad() function should ask the user for a type of salad. Then the salad() functions calls the dressing() method to asks for a dressing.

All functions **return** a string, so that you can assemble the order as a string in the calling function. The dressing() function would return the type of dressing, for example "blue cheese". The salad() function would combine the dressing and the salad type into "greek salad with lemon dressing". Finally, the select_meal() function would assemble the complete order in a string.

Note that the functions do not have any parameters. Instead, they should all read user input and then return that input formatted as a string or a list.

The following diagram illustrates the order in which the functions should be called and how the result string should be assembled from the return values step-by-step.



Some complications:

- The toppings() function should ask for toppings repeatedly until the user types "done". The return value of the toppings method should be a single string including the toppings, for example "pepperoni and mushrooms and spinach".
- The select_meal function should ask the user repeatedly to add an item until the user types "done". It should assemble a string representing the complete order.

Hints:

- None of the functions take any parameters. The important part in this problem is that you are calling functions from within other functions ("nested" function calls).
- Use string formatting with {} place holders to assemble the strings. Make sure to write all functions specified in the diagram, and only those functions with the specified behavior.

Write all your functions in a file project2.py. On the top level of project2.py (no indentation), simply call

```
select_meal()
```

to start the dialog system.